

CS528 Cloud Computing – Homework 3

Name: Vijay Krishna Sundaran Saravanan
BU ID: U63589838

1. Project Information

Project ID: cultivated-link-508221-e6

Bucket: gs://cultivated-link-508221-e6-cs528-hw2 (us-central1)

Data: 12,000 HTML files under data/, reused from Homework 2.

GitHub: <https://github.com/bu-svks1410/CS528-Cloud-Computing-Fall-2026-SVKS1410/tree/main/hw3>

The first app is hw3/cloud_function/main.py and the second is hw3/laptop_service/subscriber.py. Both were pushed to main in commit ef38987.

I deployed the first app as an HTTP cloud function. It returns the requested file for GET and POST requests, logs errors, and sends forbidden-country notifications through Pub/Sub. The second app runs on my Macbook, prints each notification and appends it to hw3-logs/forbidden-requests.jsonl in the bucket.

2. Cloud Setup and Permissions

I used my existing Google Cloud project and bucket. I ran these commands in my Mac Terminal using the account svks1410@bu.edu.

```
gcloud auth login
export PROJECT_ID="cultivated-link-508221-e6"
export BUCKET_NAME="cultivated-link-508221-e6-cs528-hw2"
export REGION="us-central1"
export SERVICE_ACCOUNT="hw3-service@${PROJECT_ID}.iam.gserviceaccount.com"
export TOPIC_ID="hw3-forbidden-requests"
export SUBSCRIPTION_ID="hw3-forbidden-requests-local"
gcloud config set project "${PROJECT_ID}"

gcloud services enable \
  cloudfunctions.googleapis.com run.googleapis.com \
  cloudbuild.googleapis.com artifactregistry.googleapis.com \
  pubsub.googleapis.com storage.googleapis.com logging.googleapis.com \
  iam.googleapis.com iamcredentials.googleapis.com \
  --project="${PROJECT_ID}"

gcloud iam service-accounts create hw3-service \
  --display-name="HW3 microservices account" --project="${PROJECT_ID}"

gcloud pubsub topics create "${TOPIC_ID}" --project="${PROJECT_ID}"

gcloud pubsub subscriptions create "${SUBSCRIPTION_ID}" \
  --topic="${TOPIC_ID}" --ack-deadline=60 --project="${PROJECT_ID}"
```

I gave the shared service account permission to read the bucket, write only under hw3-logs/, publish to the topic and receive messages from the subscription. The named user can impersonate the service account and attach it to the cloud function.

```
gcloud storage buckets add-iam-policy-binding "gs://$BUCKET_NAME" \
  --member="serviceAccount:$SERVICE_ACCOUNT" \
  --role="roles/storage.objectViewer"

export LOG_PREFIX="projects/_/buckets/${BUCKET_NAME}/objects/hw3-logs/"
gcloud storage buckets add-iam-policy-binding "gs://$BUCKET_NAME" \
  --member="serviceAccount:$SERVICE_ACCOUNT" \
  --role="roles/storage.objectUser" \
  --condition="expression=resource.name.startsWith('${LOG_PREFIX}'),title=hw3_logs_only"

gcloud pubsub topics add-iam-policy-binding "$TOPIC_ID" \
  --member="serviceAccount:$SERVICE_ACCOUNT" \
  --role="roles/pubsub.publisher" --project="$PROJECT_ID"

gcloud pubsub subscriptions add-iam-policy-binding "$SUBSCRIPTION_ID" \
  --member="serviceAccount:$SERVICE_ACCOUNT" \
  --role="roles/pubsub.subscriber" --project="$PROJECT_ID"

gcloud iam service-accounts add-iam-policy-binding "$SERVICE_ACCOUNT" \
  --member="user:svks1410@bu.edu" \
  --role="roles/iam.serviceAccountTokenCreator" --project="$PROJECT_ID"

gcloud iam service-accounts add-iam-policy-binding "$SERVICE_ACCOUNT" \
  --member="user:svks1410@bu.edu" \
  --role="roles/iam.serviceAccountUser" --project="$PROJECT_ID"
```

The service account is hw3-service@cultivated-link-508221-e6.iam.gserviceaccount.com. I tested impersonation with the following command; it completed successfully. The token was not displayed.

```
gcloud auth print-access-token \
  --impersonate-service-account="$SERVICE_ACCOUNT" \
  --project="$PROJECT_ID" > /dev/null
```

The bucket still has the public read access required by Homework 2. The country restriction is applied by the function; it does not prevent direct access to the public bucket. X-country is a client-supplied value used for this assignment.

3. How to Run Both Services

The first app uses functions-framework, google-cloud-storage and google-cloud-pubsub. The second uses google-cloud-storage, google-cloud-pubsub and google-auth. Exact versions are in each requirements.txt file.

```
git clone \
  https://github.com/bu-svks1410/CS528-Cloud-Computing-Fall-2026-SVKS1410.git
cd CS528-Cloud-Computing-Fall-2026-SVKS1410/hw3/cloud_function

gcloud functions deploy hw3-file-service \
  --gen2 --runtime=python313 --region="$REGION" \
  --project="$PROJECT_ID" --source=. --entry-point=serve_file \
  --trigger-http --allow-unauthenticated \
  --service-account="$SERVICE_ACCOUNT" \
  --set-env-vars="PROJECT_ID=$PROJECT_ID,BUCKET_NAME=$BUCKET_NAME, TOPIC_ID=$TOPIC_ID" \
  --memory=256Mi --timeout=60s --min-instances=0 --max-instances=2

export FUNCTION_URL="$(gcloud functions describe hw3-file-service \
  --gen2 --region="$REGION" --project="$PROJECT_ID" \
  --format='value(serviceConfig.uri)')"
```

Deployment completed with state ACTIVE. The function uses Python 3.13 and runs as hw3-service. Its endpoint is:

<https://hw3-file-service-cadmtlqg5a-uc.a.run.app>

To run the second app on the Mac, from the cloud_function directory:

```
cd ../laptop_service
python3 -m venv .venv
source .venv/bin/activate
python -m pip install -r requirements.txt
python -m unittest -v

python subscriber.py \
  --project=cultivated-link-508221-e6 \
  --bucket=cultivated-link-508221-e6-cs528-hw2 \
  --subscription=hw3-forbidden-requests-local \
  --service-account=hw3-service@cultivated-link-508221-e6.iam.gserviceaccount.com \
  --user-account=svks1410@bu.edu
```

I left the subscriber running in one Terminal tab and sent requests from another. Startup output confirmed the same service account as the function. Six subscriber tests passed on my Mac. Press Control+C to stop the subscriber.

4. HTTP Tests and the Provided Client

GET takes the filename from the path. POST takes it from the payload. These curl commands demonstrate the required cases:

```
export FUNCTION_URL="https://hw3-file-service-cadmtlqg5a-uc.a.run.app"
curl -sS -D get-headers.txt -o get.html "$FUNCTION_URL/data/0.html"
curl -sS -D post-headers.txt -o post.html \
  -H "Content-Type: application/json" \
  --data '{"file":"data/0.html"}' "$FUNCTION_URL/"
cmp get.html post.html
curl -i "$FUNCTION_URL/data/does-not-exist.html"
curl -i -X PUT "$FUNCTION_URL/data/0.html"
```

GET and POST returned 200 with identical contents (97,586 bytes). The missing file returned 404 Not Found, and PUT returned 501 Not Implemented. I also tested the other unsupported methods:

Method	Observed HTTP status
PUT, DELETE, OPTIONS, PATCH, HEAD	501
CONNECT	400
TRACE	405

CONNECT and TRACE are a platform limitation: Google documents that Cloud Run does not pass these methods to the service. They therefore cannot receive the function's 501 response or its application logs. The application returned 501 for all seven methods in offline routing tests, but the two public-endpoint results above do not meet the requested 501 status.

Google Cloud reference: unsupported HTTP methods

I ran the supplied macOS client for 100 requests over HTTPS. Files are numbered 0.html through 11999.html, so I used 11999 as the maximum index.

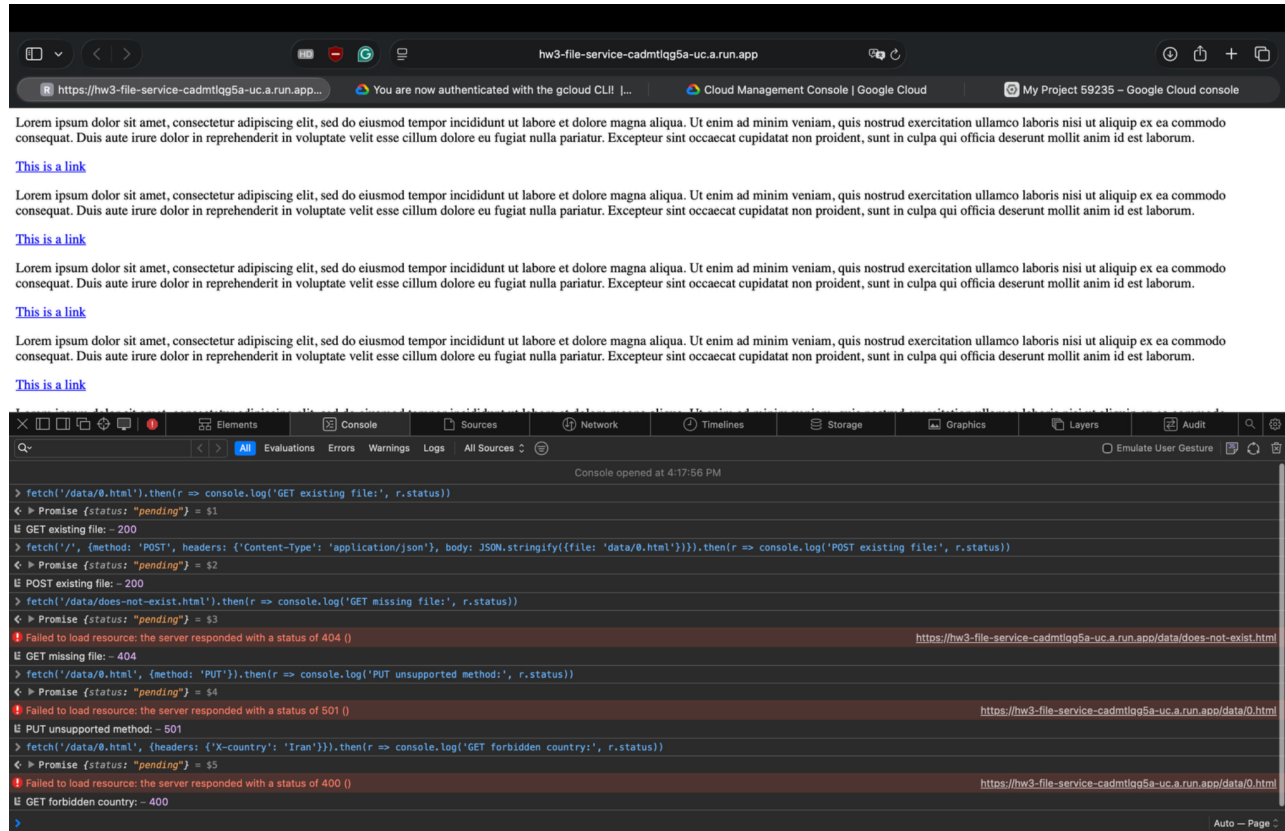
```
~/Downloads/http-client \
-d hw3-file-service-cadmtlqg5a-uc.a.run.app \
-b none -w data -n 100 -i 11999 -p 443 -s -v -r 528 -t 30 \
> client-100-requests-verbose.txt 2>&1
```

Requests	200 OK	400 forbidden country	Other statuses	Exit status
100	94	6	0	0

The six 400 responses contained "Permission denied: forbidden country". Each of their request IDs matched a structured cloud log, a plain printed cloud log and one entry in the subscriber's bucket file.

5. Browser Demonstration

I opened the function's `/data/0.html` page in Safari and used its JavaScript Console to make GET, POST and PUT requests. POST sent `{"file":"/data/0.html"}` as JSON; the forbidden-country request sent `X-country: Iran`. The screenshot shows all five results: GET 200, POST 200, missing file 404, PUT 501 and forbidden country 400.



Safari shows the actual browser requests and their returned status codes.

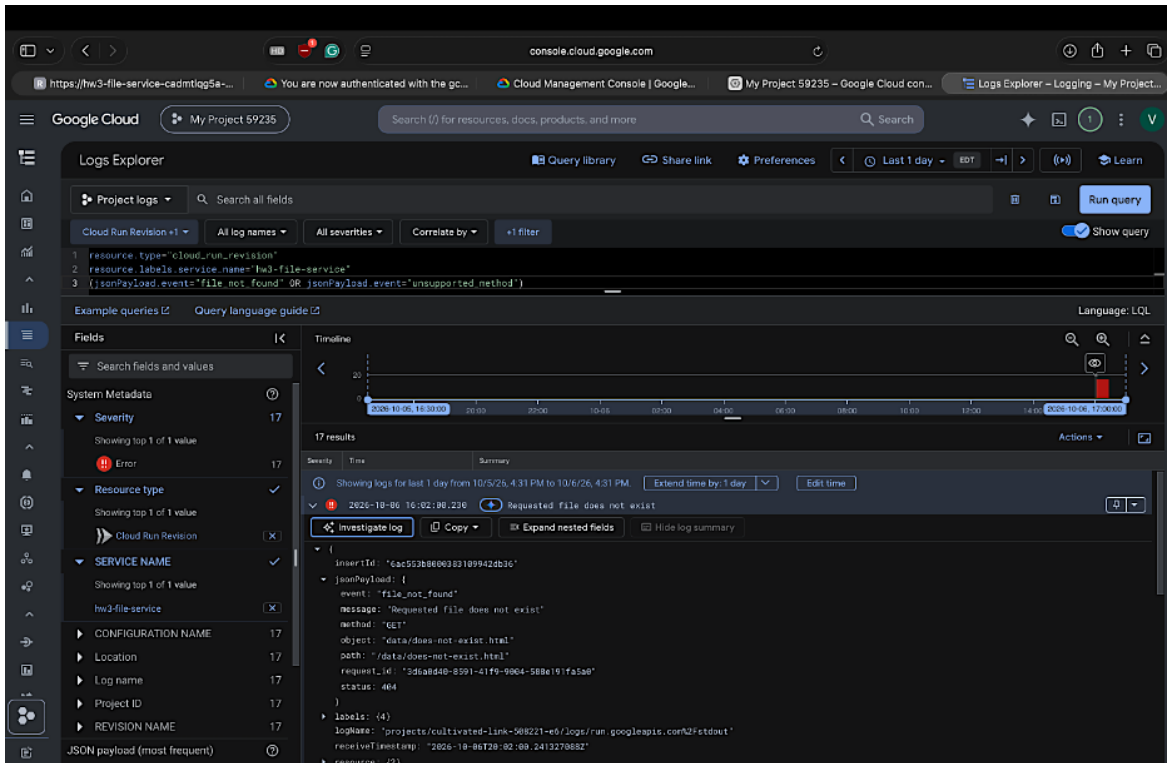
6. Cloud Logging

For 404 and 501 errors, the first app prints a JSON record and a separate plain-text error. Cloud Logging stores them as `jsonPayload` and `textPayload`. These are actual fields and text from the exported entries:

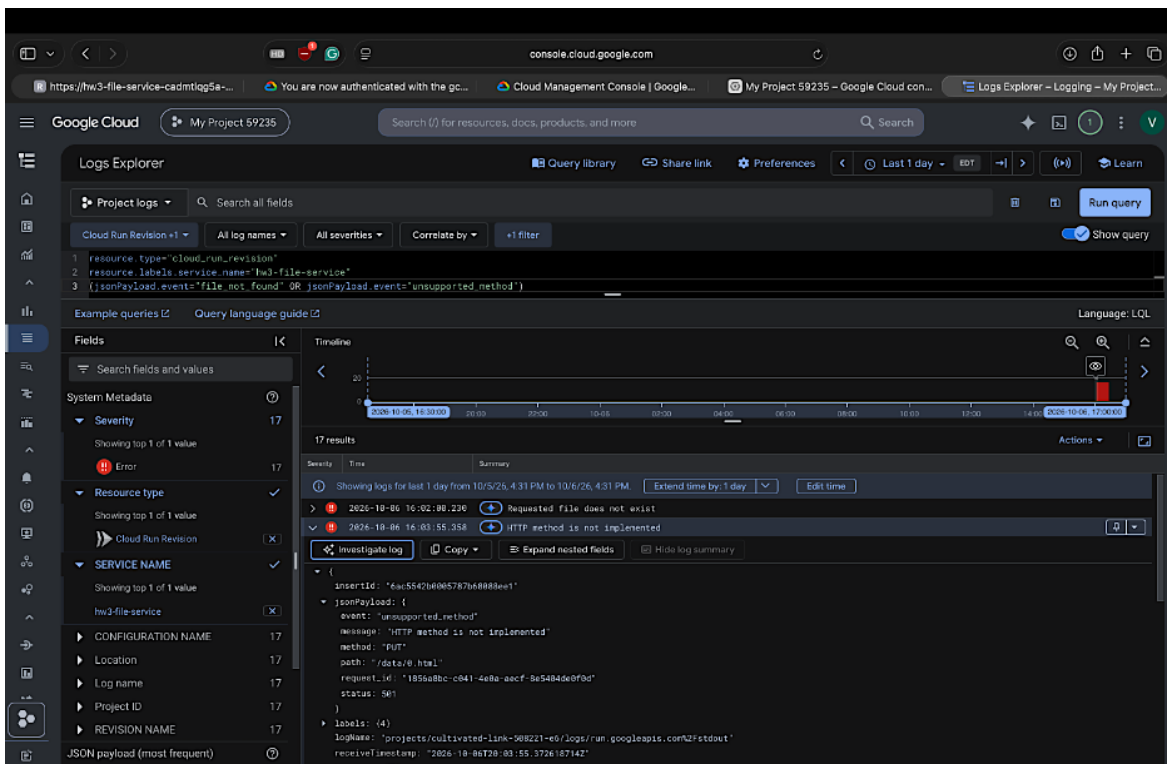
```
404 structured: event=file_not_found, method=GET, status=404
path=/data/does-not-exist.html
request_id=3d6a0d40-8591-41f9-9004-588e191fa5a0
404 plain: ERROR file_not_found: Requested file does not exist
status=404 request_id=3d6a0d40-8591-41f9-9004-588e191fa5a0

501 structured: event=unsupported_method, method=PUT, status=501
path=/data/0.html
request_id=3e0f69bb-dba2-4b7b-8bdb-ac786ea6be29
501 plain: ERROR unsupported_method: HTTP method is not implemented
status=501 request_id=3e0f69bb-dba2-4b7b-8bdb-ac786ea6be29
```

In Logs Explorer, I selected the function and expanded the error records. The screenshots show the structured 404 and 501 entries.



Cloud Logging entry for the missing file, with status 404.



Cloud Logging entry for an unsupported PUT request, with status 501.

7. Forbidden Country Requests and the Second Service

I used the assignment's list: North Korea, Iran, Cuba, Myanmar, Iraq, Libya, Sudan, Zimbabwe and Syria. The first app normalizes X-country, rejects these requests with 400, and waits for Pub/Sub to accept the notification before responding.

```
for COUNTRY in "North Korea" Iran Cuba Myanmar Iraq Libya Sudan Zimbabwe Syria; do
  curl -sS -H "X-country: $COUNTRY" -o /dev/null \
    -w "$COUNTRY: %{http_code}\n" "$FUNCTION_URL/data/0.html"
done
```

All nine countries returned 400. The laptop subscriber prints the error and appends the event to hw3-logs/forbidden-requests.jsonl. It reads the existing log and uploads the old content plus the new line. A generation check prevents overwriting a concurrent update. It acknowledges the Pub/Sub message after saving and checks message IDs to avoid duplicate lines on redelivery.

```
ERROR: Permission denied: request from forbidden country iran
Saved: gs://cultivated-link-508221-e6-cs528-hw2/hw3-logs/forbidden-requests.jsonl
```

The exported file contained 23 notifications from all tests, with 23 unique Pub/Sub message IDs. All nine countries were present. This included all six forbidden requests from the recorded 100-request run. The initial Iran response and saved record both used request ID 0665db32-8d32-456d-b073-b7c51012cedc.

To read the saved file:

```
gcloud storage cat \
  gs://cultivated-link-508221-e6-cs528-hw2/hw3-logs/forbidden-requests.jsonl
```

8. Service Account Authentication on the Laptop

I chose service account impersonation without an account key. I signed in to the CLI with gcloud auth login. The program uses the following command internally to obtain a temporary token for hw3-service, using the Token Creator permission granted to svks1410@bu.edu:

```
gcloud auth print-access-token \
  --impersonate-service-account="$SERVICE_ACCOUNT" \
  --project="$PROJECT_ID" --account=svks1410@bu.edu \
  --lifetime=3600 --quiet
```

The Python program captures the token and passes an explicit credential object to the Storage and Pub/Sub clients. Their requests therefore use the same service account as the cloud function. It requests a one-hour token and conservatively refreshes it after about 45 minutes through the credential library. Tokens are held in memory and are not printed or put in GitHub.

I chose this mechanism to avoid storing a long-lived private key on the laptop. The user's CLI login is only used to obtain the impersonated token; the applications access cloud resources as hw3-service. I did not use gcloud auth application-default login or download a service account key.

[Google Cloud reference: temporary access tokens and impersonation](#)